

PHASE 4 · GENERATIVE AI · WEEK 12

RAG Pipeline

Retrieval-Augmented Generation: grounding LLMs in your own documents

Vector databases (ChromaDB)

Chunking strategies

Retrieval & reranking

End-to-end RAG app

◆ **Project: RAG chatbot over a PDF knowledge base**

THE PROBLEM

LLMs only know what they were trained on



Knowledge cutoff

Anything newer than the training data is invisible to the model.



Private data

Your PDFs, wikis and tickets were never in the training set.



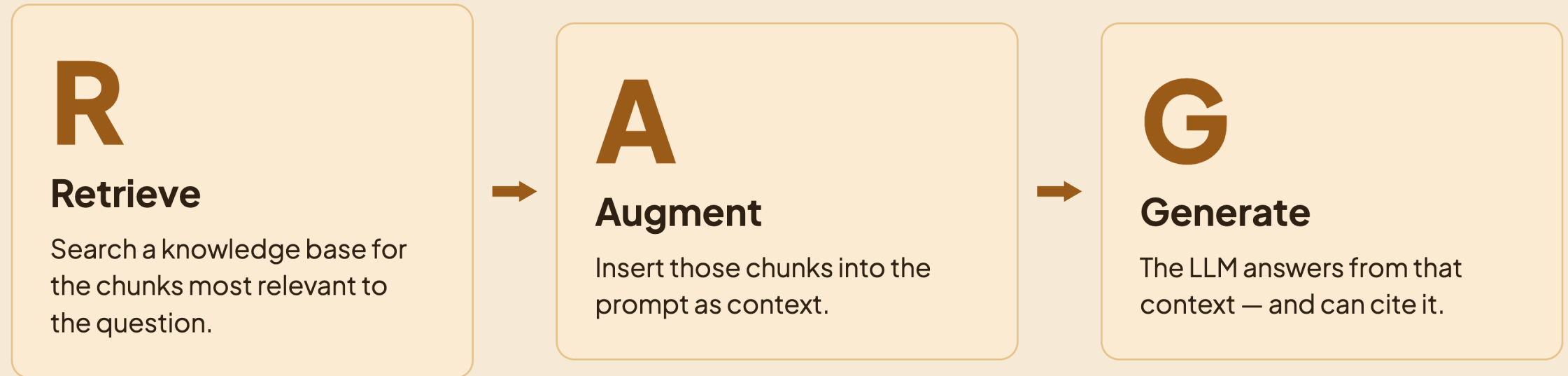
Hallucination

Asked about unknowns, models often answer fluently — and wrongly.

Fine-tuning teaches style and skills. **RAG supplies facts** — fresh, private and citable — at question time.

THE IDEA

Retrieve, augment, generate

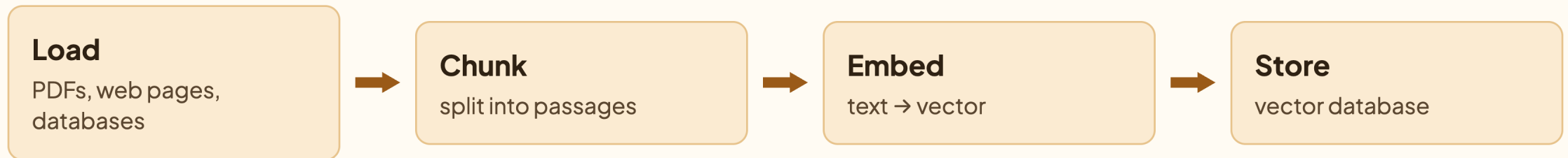


Think open-book exam: the model doesn't memorise the textbook, it looks things up.

ARCHITECTURE

Two phases: index once, query many times

INDEXING · OFFLINE



QUERYING · ONLINE



Both phases must use the **same embedding model** — vectors from different models aren't comparable.

RULE OF THUMB

Most RAG failures are retrieval failures.

If the right chunk never reaches the prompt, no LLM can answer correctly. Debug retrieval first.

Chunking strategies

Strategy	How it splits	Good for	Watch out for
Fixed-size	Every N characters or tokens, with overlap	Quick baselines	Cuts sentences mid-thought
Recursive	Paragraphs → sentences → words, until under the size limit	The default for most text	Still blind to document structure
Document-aware	On headings, sections, pages or code blocks	Markdown, HTML, manuals	Section lengths vary wildly
Semantic	Where meaning shifts between sentences (embedding similarity drops)	Prose that changes topic	Slower; extra embedding cost

Chunk size and overlap are trade-offs

Smaller chunks

- Precise matches
- Less noise in the prompt
- But: lose surrounding context

Larger chunks

- Keep related ideas together
- Fewer chunks to search
- But: blurrier embeddings, more tokens

Overlap

Repeats the end of one chunk at the start of the next, so an idea cut at a boundary still appears whole somewhere.

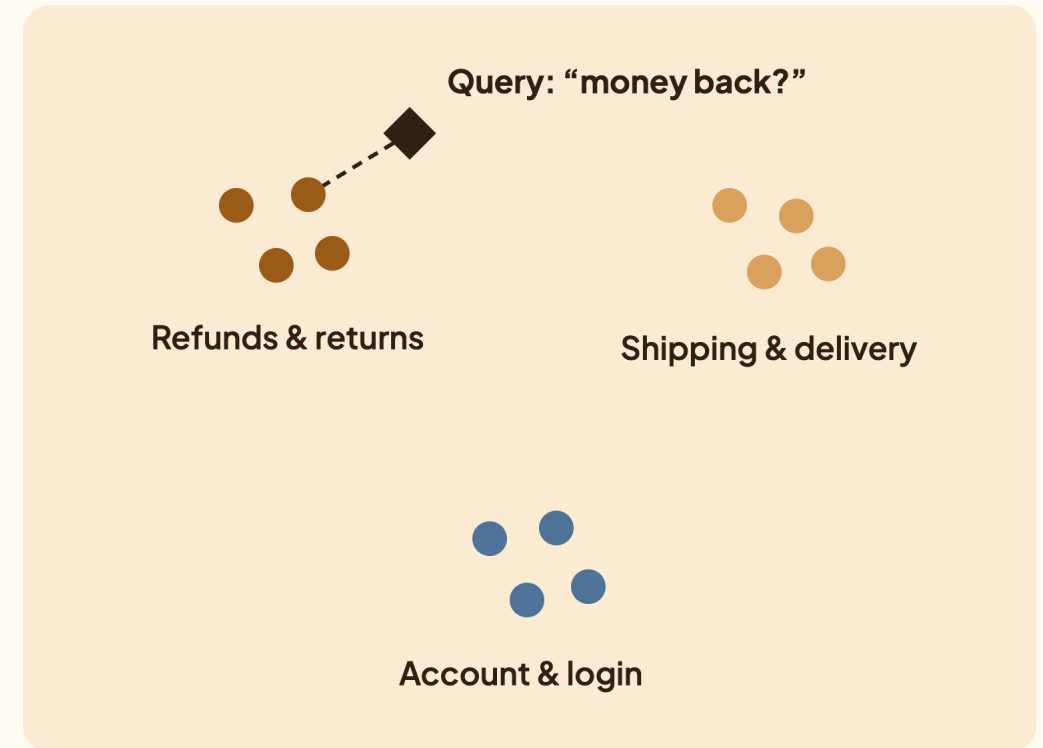
Common starting point: ~1,000 characters with ~200 overlap — then tune on your own questions.

Embeddings turn meaning into geometry

An embedding model maps text to a vector — **all-MiniLM-L6-v2** outputs 384 numbers. Texts with similar meaning land close together, even with no words in common.

Cosine similarity compares directions: near 1 = very similar, near 0 = unrelated.

Retrieval = embed the question, return its nearest neighbours.



VECTOR DATABASES

Storing and searching vectors with ChromaDB

-  Stores vectors with the original text and metadata
-  Finds nearest neighbours fast with approximate search (HNSW)
-  Filters by metadata, e.g. source file or page
-  Persists to disk, so you index once and reuse

```
import chromadb
client = chromadb.PersistentClient(path="./db")
col = client.get_or_create_collection("handbook")

col.add(
    ids=["p1-c0", "p1-c1"],
    documents=[chunk0, chunk1],
    metadatas=[{"page": 1}, {"page": 1}],
)

res = col.query(
    query_texts=["How do refunds work?"],
    n_results=3,
)
```

Chroma embeds text with a default model unless you pass your own embedding function.

Retrieval: finding the right chunks

Dense (semantic)

Nearest neighbours in embedding space. Catches paraphrases.

Sparse (BM25)

Keyword scoring. Nails exact terms, IDs and names.

Hybrid

Merge both rankings, e.g. reciprocal rank fusion. A strong default.

Metadata filters

Restrict by source, date, page or user before searching.

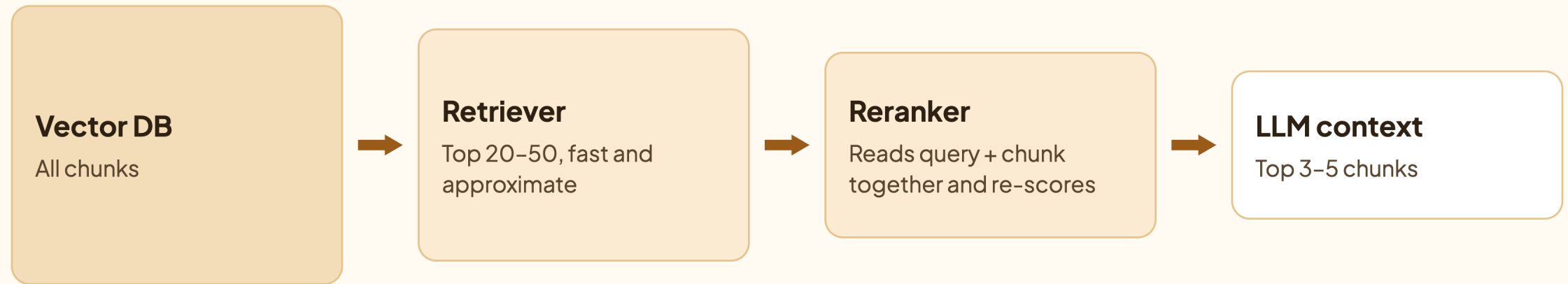
MMR

Trade a little relevance for diversity to avoid near-duplicate chunks.

Query rewriting

Rephrase, expand or split the question first (multi-query, HyDE).

Rerank: retrieve wide, then keep the best



Bi-encoder (retriever): embeds query and chunk separately — fast and precomputable, less precise.

Cross-encoder (reranker): scores each query-chunk pair jointly — slower, more accurate. e.g. ms-marco-MiniLM-L-6-v2

The augmented prompt

SYSTEM

Answer using ONLY the context below.
If it isn't there, say "I don't know."
Cite sources as [page N].

CONTEXT

[page 4] Refunds are issued within ...
[page 9] Items must be returned ...

QUESTION

Can I get a refund after 20 days?

Ground it

Tell the model to answer only from the context.

Allow “I don't know”

Reduces confident guessing when retrieval misses.

Cite sources

Chunk metadata (file, page) lets users verify answers.

EVALUATION

Evaluate retrieval and generation separately

Retrieval

Hit rate @ k

Is a relevant chunk in the top k?

Recall @ k

What share of the relevant chunks came back?

MRR

How high does the first relevant chunk rank?

Generation

Faithfulness

Is every claim supported by the context?

Answer relevance

Does it actually answer the question?

Correctness

Does it match a reference answer?

Build a small test set of real questions with the pages that answer them. Tools: Ragas, or an LLM-as-judge (Week 10).

DEBUGGING

Common failures and fixes

Symptom	Try
The answer exists but isn't retrieved	Tune chunk size, add hybrid search, rewrite the query
Retrieved, but the answer is still wrong	Rerank, put the best chunks first, tighten the prompt
Confident answer with no support	Require citations, allow “I don't know”, check faithfulness
Tables and scanned pages come out garbled	Use a layout-aware parser or OCR; read your chunks
Stale answers after documents change	Re-index on update; keep doc IDs and versions in metadata

PROJECT

RAG chatbot over a PDF knowledge base

- 1 **Load** — PDFs with pypdf or PyPDFLoader; keep file and page numbers
- 2 **Chunk** — RecursiveCharacterTextSplitter; inspect a few chunks
- 3 **Embed & store** — all-MiniLM-L6-v2 vectors in a persistent ChromaDB
- 4 **Retrieve & rerank** — Top 20 from Chroma → cross-encoder → keep top 4
- 5 **Generate** — Grounded prompt with [file, page] citations
- 6 **Converse** — Week 11 memory; rewrite follow-ups into standalone questions

Done when it...

- Answers cite file and page
- Says “I don't know” when the PDFs don't cover it
- Handles follow-ups like “what about section 3?”
- Scored on your own test questions

Stretch: hybrid search · Gradio UI · upload new PDFs live

RECAP

Week 12 in five lines

- 1** RAG grounds an LLM in your documents at question time.
- 2** Indexing: load → chunk → embed → store. Chunking shapes everything downstream.
- 3** Querying: retrieve wide, rerank, and pass only the best context.
- 4** Prompt for grounding, citations and “I don't know”.
- 5** Measure retrieval and generation separately — fix retrieval first.